

501: Linux Operating System [2511000905044001]

Unit 2: Basic Linux Commands

- 2.1 Directory Navigation Commands (pwd, cd, mkdir, rmdir, ls, tree)
- 2.2 File Management Commands (cat, rm, cp, mv, touch)
- 2.3 File Permissions and Ownership (chmod, chgrp, chown, umask)
- 2.4 Common System Commands (who, whoami, man, echo, date, clear)
- 2.5 Text Processing Commands (head, tail, cut, sort, cmp, tr, uniq, wc, tee)
- 2.6 Introduction to Process
- 2.7 Process Control commands : ps, fg, bg, kill, sleep
- 2.8 Job Scheduling commands : at, batch, crontab

Directory Navigation Commands (pwd, cd, mkdir, rmdir, ls, tree):

1. pwd – Print Working Directory

- Shows the full path of the current working directory.
- Helps to know your current location in the file system.

Examples:

```
bash  
pwd
```

Output:

```
/home/student/Documents
```

This means you are in the Documents directory under /home/student.

2. cd – Change Directory

- Used to move from one directory to another.
- You can go to any folder using its path.
- Use cd .. to go one level up.
- cd ~ goes to the home directory.
- cd / moves to the root directory.

Examples:

```
Bash
```

```
cd /home/student/Documents    # Go to Documents
```

```
cd ..          # Go one directory up  
cd ~           # Go to home directory  
cd /           # Go to root directory  
cd /var/log    # Go to log folder
```

3. mkdir – Make Directory

- Used to create a new directory (folder).
- Use to organize files in folders.
- Use **-p** to create multiple nested directories.

Examples:

Bash

```
mkdir myfolder          # Create a folder named myfolder  
mkdir College/Notes     # Error if "College" doesn't exist  
mkdir -p College/Notes/Unit1  # Creates entire path
```

4. rmdir – Remove Directory

- Deletes empty directories.
- Works only on empty folders.
- Use **rm -r** for non-empty folders (be careful!).

Examples:

bash

```
rmdir myfolder          # Removes the folder if empty  
rmdir College/Notes/Unit1  # Only if Unit1 is empty
```

To remove a folder with files inside:

Bash

```
rm -r myfolder          # DANGEROUS: deletes everything inside
```

5. ls – List Files and Directories

- Shows the contents of a directory.
- Use **ls** to view files/folders.
- Use **-l** for detailed view.
- Use **-a** to show hidden files (files starting with .).

Examples:

bash

```
ls                # List items in current folder
ls -l             # Long listing with details
ls -a             # Show hidden files
ls /etc           # List contents of /etc directory
```

6. tree – Directory Tree View

- Shows a hierarchical (tree-like) structure of directories and files.
- Great for visual understanding of directory structure.
- May need to install it using `sudo apt install tree`.

Examples:

Bash

```
tree              # Show tree of current directory
tree /home/student # Show tree of specific path
tree -L 2         # Limit depth to 2 levels
```

Example Output:

```
.
├── College
│   ├── Notes
│   │   └── Unit1
│   └── Assignments
└── file.txt
```

Command	Full Form	Purpose	Key Options
pwd	Print Working	Show current path	None

Command	Full Form	Purpose	Key Options
	Directory		
cd	Change Directory	Move to another directory	.. , ~, /
mkdir	Make Directory	Create new directory	-p
rmdir	Remove Directory	Delete empty folder	(only empty),-r
ls	List	List contents	-l, -a (start with . file)
tree	Tree View	Visualize directory structure	-L <level>

File Management Commands (cat, rm, cp, mv, touch)

1. cat – View or create a file

- Used to create, view, and concatenate files.

Common uses:

- View content of a file
- Create a new file
- Join multiple files

Syntax:

cat [options] [filename]

Examples:

cat file.txt	# Display contents of file.txt
cat > newfile.txt	# Create a new file and add content (press Ctrl+D to save)
cat file1.txt file2.txt	# Display contents of both files
cat file1.txt file2.txt > combined.txt	# Merge two files into one

2. rm – Remove (delete) files or directories

Deletes files or directories permanently (△□ No recycle bin).

Common uses:

- Delete a file
- Delete a directory

Syntax:

rm [options] [filename]

Examples:

rm file.txt	# Delete file.txt
rm -i file.txt	# Ask before deleting (interactive mode)
rm *.txt	# Delete all .txt files in current folder
rm -r myfolder	# Delete a folder and its contents (recursive)
rm -rf myfolder	# Force delete a folder and contents without prompt (DANGEROUS)

3. cp – Copy files and folders

Makes a copy of files or directories.

Common uses:

- Copy files or folders to another location

Syntax:

cp [options] source destination

Examples:

cp file.txt backup.txt	# Copy file.txt as backup.txt
cp file.txt /home/user/Documents/	# Copy file to another directory
cp -r folder1/ folder2/	# Copy folder1 and all its contents to folder2

4. mv – Move or rename files/folders

Moves files/folders or renames them.

Common uses:

- Rename a file or folder
- Move file to another directory

Syntax:

`mv [options] source destination`

Examples:

```
mv oldname.txt newname.txt      # Rename file
mv file.txt /home/user/Desktop/  # Move file to Desktop
mv folder1 /home/user/Documents/ # Move folder1 to Documents
```

5. touch – Create a new empty file or update timestamp

Creates an empty file or updates last modified time of a file.

Common uses:

- Create new file
- Update file timestamp

Syntax:

`touch [filename]`

Examples:

```
touch newfile.txt      # Create an empty file
touch file1.txt file2.txt # Create multiple files at once
touch -c notexisting.txt # Do nothing if file doesn't exist (no error)
```

Command	Use	Example
Cat	View or create file	cat file.txt
Rm	Delete file/folder	rm file.txt, rm -r folder
Cp	Copy file/folder	cp a.txt b.txt
Mv	Move or rename	mv old.txt new.txt
Touch	Create empty file	touch file.txt

File Permissions and Ownership (chmod, chgrp, chown, umask)

Every file and directory in Linux has:

- **User (u):** Owner of the file
- **Group (g):** Group that owns the file
- **Others (o):** All other users

Each can have:

- r = read
- w = write
- x = execute

1. chmod – Change File Mode (Permissions)

Full Form:

chmod = Change Mode

Use:

- Change read, write, execute permissions of a file or folder

Syntax:

chmod [options] mode filename

Methods:

1. Symbolic Method:

```
chmod u+x script.sh # Add execute to user
chmod g-w notes.txt # Remove write from group
chmod o=r file.txt  # Set read-only for others
```

2. Octal (Numeric) Method:

```
chmod 755 file.sh # rwxr-xr-x
chmod 644 file.txt # rw-r--r--
```

Number	Permission
7	rwx
6	rw-
5	r-x
4	r--
3	-wx
2	-w-
1	--x
0	---

2. chgrp – Change Group Ownership

Full Form:

chgrp = Change Group

Use:

- Change the **group** ownership of a file or folder

Syntax:

Chgrp groupname filename

Example:

```
chgrp developers report.txt # Assigns group 'developers' to report.txt
ls -l                      # To check new group owner
```

3. chown – Change Owner and Group

Full Form:

chown = Change Ownership

Use:

- Change **owner** and optionally **group** of a file

Syntax:

chown owner filename
chownowner:group filename

Examples:

chown john file.txt # Change owner to john
chownjohn:devs project.txt # Change owner to john and group to devs

4. umask – User File-Creation Mode Mask

Full Form:

umask = User Mask

Use

- Sets **default permissions** for new files and folders

Syntax:

umask # View current umask
umask 022 # Set new default mask

How It Works:

- Default permission for files: **666 (rw-rw-rw-)**
- Default permission for folders: **777 (rwxrwxrwx)**
- Subtract umask from default

Example:

umask 022 → file gets 644 (rw-r--r--)

Command	Full Form	Function	Example
chmod	Change Mode	Change file/folder permissions	chmod 755 script.sh
chgrp	Change Group	Change group ownership	chgrp students notes.txt
chown	Change	Change file owner and	chownalice:staff

Command	Full Form	Function	Example
	Ownership	group	doc.pdf
umask	User Mask	Set default permissions for new files	umask 027

Common System Commands (who, whoami, man, echo, date, clear)

1. who – Who is Logged In

Full Form:

who = Who is logged into the system

Use:

Displays information about users currently logged into the system.

Syntax:

who

Example:

who

Output Example:

```
student  tty7  2025-07-04 08:30
```

Explanation:

- student – Username
- tty7 – Terminal
- 2025-07-04 08:30 – Login date and time

2. whoami – Who Am I

Full Form:

whoami = Who am I (current user)

Use:

Shows the **currently logged-in username**.

Syntax:

whoami

Example:

whoami

Output:

student

3. man – Manual

Full Form:

man = Manual

Use:

Shows the **manual/help page** for any Linux command.

Syntax:

man [command_name]

Example:

man ls # Shows manual for 'ls' command

Use **Up/Down arrows** to scroll, **q** to quit.

4. echo – Display a Message

Full Form:

echo = Repeat/display text or variables

Use:

- Display text on screen
- Show variable values

- Print output to files

Syntax:

echo [text or variable]

Examples:

```
echo Hello Students      # Display a message
echo $HOME               # Show the current user's home directory
echo "My name is $USER"  # Show current username
```

5. date – Display Date and Time

Full Form:

date = Current system date and time

Use:

- Show or set current date and time

Syntax:

date

Example:

date

Output:

Thu Jul 4 20:10:45 IST 2025

Format option:

date +"%d-%m-%Y %H:%M:%S"

6. clear – Clear the Terminal

Full Form:

clear = Clear screen

Use:

Clears the terminal screen for better visibility.

Syntax:

clear

Example:

clear

The terminal becomes empty (old text is pushed up/out of view).

Command	Full Form	Purpose	Example
who	Who is logged in	Show logged-in users	who
whoami	Who am I	Show current username	whoami
man	Manual	Help/manual for commands	man cp
echo	Display Text	Print message or variable	echo \$HOME
date	Date & Time	Show current date/time	date +"%d-%m-%Y"
clear	Clear Screen	Clear the terminal display	clear

Text Processing Commands (head, tail, cut, sort, cmp, tr, uniq, wc, tee)**1. head – Display Beginning of File****Full Form:**

head = Show top lines

Use:

Displays the **first 10 lines** of a file (default), or a specific number.

Syntax:

head [options] filename

Examples:

```
head data.txt          # Show first 10 lines
head -n 5 marks.txt    # Show first 5 lines
```

2. tail – Display End of File

Full Form:

tail = Show bottom lines

Use:

Displays the **last 10 lines** of a file (default), or a specific number.

Syntax:

```
tail [options] filename
```

Examples:

```
tail data.txt          # Show last 10 lines
tail -n 3 marks.txt    # Show last 3 lines
```

3. cut – Cut Sections from Each Line

Full Form:

cut = Cut selected portions of text

Use:

Extracts **columns** or **fields** from a file or input.

Syntax:

```
cut -c column_range filename # For characters
cut -d 'delimiter' -f field_no filename # For fields
```

Examples:

```
cut -c 1-5 names.txt      # Show first 5 characters of each line
cut -d ',' -f 1 students.csv # Show first field from CSV
```

4. sort – Sort Text Data

Full Form:

sort = Sort lines

Use:

Sorts lines in a file **alphabetically or numerically**.

Syntax:

sort [options] filename

Examples:

```
sort names.txt           # Sort alphabetically
sort -r names.txt        # Reverse sort
sort -n marks.txt        # Sort numbers
```

5. cmp – Compare Two Files

Full Form:

cmp = Compare

Use:

Compares **two files byte by byte**.

Syntax:

cmp file1 file2

Examples:

```
cmp a.txt b.txt          # Output first difference
```

Output Example:

a.txt b.txt differ: byte 5, line 1

6. tr – Translate or Replace Characters

Full Form:

tr = Translate

Use:

Used to **translate**, **delete**, or **squeeze** characters from input.

Syntax:

tr [options] SET1 SET2

Examples:

```
echo "hello" | tr a-z A-Z    # Convert to uppercase
echo "121-333" | tr -d '-'   # Delete '-'
```

7. uniq – Filter Duplicate Lines

Full Form:

uniq = Unique lines

Use:

Removes **repeated lines** in sorted files.

Syntax:

uniq [options] filename

Examples:

```
uniq names.txt          # Remove duplicates (must be sorted first)
sort names.txt | uniq    # Best practice
uniq -c names.txt        # Count duplicates
```

8. wc – Word Count

Full Form:

wc = Word Count

Use:

Counts **lines, words, characters**, or **bytes**.

Syntax:

wc [options] filename

Examples:

```
wc file.txt          # Show lines, words, characters
wc -l file.txt       # Line count only
wc -w file.txt       # Word count
```

Output Example:

```
10 50 300 file.txt
```

9. tee – Redirect Output and Display

Full Form:

tee = T-shaped pipe (split output)

Use:

Saves command output to **file and screen at the same time**.

Syntax:

command | tee filename

Examples:

```
ls | tee list.txt      # Save and display output
date | tee -a log.txt  # Append date to file
```

Command	Full Form	Purpose	Example
head	Show top lines	Display first lines of file	head file.txt
tail	Show bottom lines	Display last lines of file	tail -n 5 data.txt

Command	Full Form	Purpose	Example
cut	Cut	Extract columns or fields	cut -d ',' -f1 data.csv
sort	Sort	Sort file contents	sort names.txt
cmp	Compare	Compare two files	cmp file1.txt file2.txt
tr	Translate	Replace or delete characters	tr a-z A-Z
uniq	Unique	Remove duplicate lines	uniq data.txt
wc	Word Count	Count lines, words, and characters	wc -l file.txt
tee	T-pipe redirect	Save and show command output	`ls

Introduction to Process in Linux :

What is a Process?

- A process is a program in execution.
- Every process has a unique PID (Process ID).

Types of Processes

- Foreground Process: Runs directly under user control in the terminal.
- Background Process: Runs without user interaction, often denoted with &.

States of a Process

- Running
- Sleeping
- Stopped
- Zombie

1. Running (R)

- The process is **actively running** on the CPU or **ready to run**.
- It's either executing or waiting to be assigned to a CPU core.
- Example: A program calculating values.

2. Sleeping (S / D)

- The process is **not active** but **waiting** for some **event** (like input/output).
- Two types:
 - **Interruptible Sleep (S)**: Can be awakened by signals (e.g., waiting for user input).
 - **Uninterruptible Sleep (D)**: Cannot be interrupted easily (e.g., waiting for disk I/O).

3. Stopped (T)

- The process is **halted** (paused) by a signal.
- It can be resumed later.
- Example: When you press Ctrl+Z to pause a foreground process.

4. Zombie (Z)

- The process has **finished execution**, but its **parent has not read** its exit status.
- It still appears in the process table.
- It's a "dead" process waiting to be **cleaned up** by its parent using `wait()`.

Summary Table:

State	Code	Description
Running	R	Actively using CPU or ready to run
Sleeping	S/D	Waiting for event (interruptible/uninterruptible)
Stopped	T	Process paused, waiting to be resumed
Zombie	Z	Process completed, waiting for cleanup

1. Parent and Child Process

- Processes are created using `fork()` (in system programming).
- The original is the parent, the new one is the child.

2. init/systemd Process

- First process started during boot.
- PID = 1 and responsible for starting other processes.

3. Process Table

- Maintains all information about active processes.

Process Control Commands

1. **ps** – Display Process Status

- Shows current running processes.

Example:

```
ps
ps -aux          # Show all running processes in detail
```

2. **fg** – Bring a background job to foreground

Resumes a background job in the foreground.

Example:

```
fg %1           # Bring job 1 to foreground
```

3. **bg** – Resume a job in the background

Moves a stopped or suspended job to the background.

Example:

```
bg %1           # Resume job 1 in background
```

4. **kill** – Terminate a process

Sends a signal (default is SIGTERM) to terminate a process.

Example:

```
kill 1234        # Kill process with PID 1234
kill -9 1234     # Forcefully kill (SIGKILL)
```

5. **sleep** – Delay execution

Pauses a process for a given time in seconds.

Example:

```
sleep 5          # Waits for 5 seconds
```

Job Scheduling in Linux

Job scheduling allows you to run commands or scripts automatically at a specific time or event.

Linux provides commands like ``at``, ``batch``, and ``crontab`` for this purpose.

1. ``at`` Command (One-time Scheduling)

Run a command once at a specific time in the future.

Syntax:

`at TIME`

Steps to Use:

- Type `at` followed by time.
- Enter your command(s).
- Press `Ctrl + D` to finish.

Example:

`at 3:00 PM`

After entering this, type:

```
echo "Time to study!" >> message.txt
```

Then press `Ctrl + D`.

This will create a file `message.txt` with the message at 3:00 PM.

View Scheduled Jobs:

```
atq
```

Remove a Job:

atrm <job_number>

2. batch Command (Run When System Load Is Low)

Purpose:

Run commands **once** but **only when the system load is low**.

Syntax:

batch

Example:

batch

Then type:

sh backup.sh

Then press Ctrl + D.

This will run backup.sh when the system is not busy.

3. crontab Command (Recurring Scheduling)

Purpose:

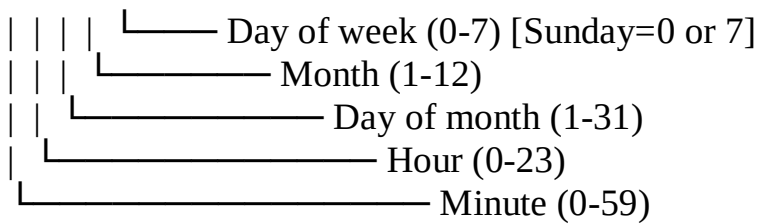
Run commands **automatically at regular intervals** (daily, weekly, monthly, etc.).

Syntax:

```
Crontab -e    # Edit the crontab
crontab -l    # List current crontab
crontab -r    # Remove current crontab
```

Crontab Time Format:

```
sql
* * * * * command_to_run
| | | | |
```



Examples:

Schedule	Crontab Entry	Description
Every day at 6 AM	0 6 * * * echo "Good morning!"	Prints message daily at 6 AM
Every Monday at 9 AM	0 9 * * 1 sh weekly_report.sh	Runs script every Monday at 9 AM
Every 5 minutes	*/5 * * * * date >> time.log	Appends date to file every 5 minutes
On 1st of every month at 10 PM	0 22 1 * * sh monthly.sh	Runs script on the 1st of each month at 10 PM

View Current Jobs:

crontab -l

Command	Use for	Runs at...	Useful When
at	One-time tasks	Specific future time	Reminder, one-time backup
batch	One-time tasks (low load)	When system is idle	Heavy scripts (e.g., backup)
crontab	Repeated tasks	Fixed intervals	Automation (reports, emails)

Steps to Use:

1. Type `at` followed by time.
2. Enter your command(s).
3. Press `Ctrl + D` to finish.

Example:

at 3:00 PM

After entering this, type:

```
echo "Time to study!" >> message.txt
```

Then press `Ctrl + D`.

This will create a file `message.txt` with the message at 3:00 PM.

View Scheduled Jobs:

```
atq
```

Remove a Job:

```
atrm <job_number>
```

2. `batch` Command (Run When System Load Is Low)

Purpose:

Run commands ****once**** but ****only when the system load is low****.

Syntax:

```
batch
```

Example:

```
batch
```

Then type:


```
sh backup.sh
```

Then press `Ctrl + D`.

This will run `backup.sh` when the system is not busy.

3. `crontab` Command (Recurring Scheduling)

Purpose:

Run commands automatically at regular intervals (daily, weekly, monthly, etc.).

Syntax:

```
crontab -e    # Edit the crontab
```

```
crontab -l    # List current crontab
```

```
crontab -r    # Remove current crontab
```

Crontab Time Format:

command_to_run

```
| | | | |
```

```
| | | |  _____ Day of week (0-7) [Sunday=0 or 7]
```

```
| | |  _____ Month (1-12)
```

```
| |  _____ Day of month (1-31)
```

```
|  _____ Hour (0-23)
```

```
_____ Minute (0-59)
```

View Current Jobs:

```
crontab -l
```

Make sure `atd` service is running to use `at` and `batch`:

```
sudo systemctl start atd
```